


```
// — DIGIT EXTRACTION LOOP — peel digits
right-to-left
while (n != 0) {
    int digit = n % 10;
    n /= 10;
}

// — FAST POWER (exponentiation by
// squaring) —
double fastPow(double x, long n) {
    double result = 1;
    while (n > 0) {
        if ((n & 1) == 1) result *= x;
        x *= x;
        n >>= 1;
    }
    return result;
}

// — BASE CONVERSION — generic base b,
// 0-indexed
StringBuilder builder = new
StringBuilder();
while (num > 0) {
    builder.append(num % b);
    num /= b;
}
builder.reverse();
int value = 0;
for (int digit : digits) value = value * b
+ digit;

// — SIEVE OF ERATOSTHENES — mark
// composites up to n
boolean[] composite = new boolean[n];
for (int i = 2; i * i < n; i++) {
    if (composite[i]) continue;
    for (int j = i * i; j < n; j += i)
        composite[j] = true;
}
```